

PYTHON

Module 18: Conditional Control Flow

CODING

Logical Decision Making

Control flow is the core of smart programming. In Python, we use 'Conditional Statements' to decide which sections of code should run based on logic.

Python supports standard mathematical conditions:

- ****Equality****: `a == b` and `a != b`



- ****Comparison****:

`a < b` , `a <= b` , `a > b` , `a >= b`



These expressions evaluate to Booleans (True or False), which then trigger specific code blocks.



The Indentation Rule

Python is unique because it uses white space (indentation) to define the scope of a code block. While other languages use curly brackets {}, Python requires consistent indentation.

****Crucial Tip**:** An if statement without indented code below it will raise an `IndentationError`. All lines within the same 'action block' must be indented at the exact same level.

Multi-Path Logic: elif & else

Rarely is a decision a simple 'Yes' or 'No'. Python provides the `elif` and `else` keywords to handle multiple outcomes.

- **`**if**`**: The initial check.

• **`**elif**`**: Short for 'else if'. Runs only if the previous conditions were False.

• **`**else**`**: The final 'catch-all'. It executes if none of the preceding conditions were met.

`**Efficiency Note**`: Python stops checking conditions as soon as it finds the first `True` match in a chain.

Shorthand & Ternary Operators

For simple logic, Python allows 'Conditional Expressions' that fit on a single line.

• ****Short Hand If****: `if a > b: print("A")`

• ****Ternary Operator****: `print("A") if a > b else print("B")`

At CodeHive, we use shorthand for simple variable assignments to keep the code concise and readable.

Logical Operators (and, or, not)

To build complex filters, we combine conditions using logical operators:

- ****and****: Returns True if both statements are True.

- ****or****: Returns True if at least one statement is True.

- ****not****: Reverses the current result (True becomes False).

Example: `if age > 18 and has_ticket: print("Entry allowed")`

Nested Logic

You can place an `if` statement inside another `if` statement. This is known as 'Nesting'.

While powerful, deep nesting can make code hard to read. CodeHive recommends using logical operators (`and`) to flatten logic whenever possible, though nesting is necessary for sequential dependency checks.

The 'pass' Statement

In Python, an `if` block cannot be empty. If you are designing the structure of a program but haven't written the logic yet, use the `pass` statement.

It acts as a placeholder that does absolutely nothing, but it prevents the code from crashing with a syntax error during development.